Analyze A/B Test Results

This project analyzes the results of an A/B test run by an e-commerce website.
Goal: Determine whether to implement the new (treatment) page, keep the old (control) page, or run the experiment longer.

Table of Contents

- Part I – Descriptive Statistics
- Part II – Probability
- Part III – Experimentation
- Standout – Expanded EDA & Logistic Regression

Imports

In [1]:
```
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder

random.seed(0)
np.random.seed(0)
```

Part I – Descriptive Statistics

`a)` Read in the `ab_data.csv` dataset and inspect the first few rows.

In [2]:
```
df = pd.read_csv('ab_data.csv')
df.head()
```

Out[2]:

	country	group	converted
0	UK	control	0
1	US	treatment	1
2	UK	treatment	0
3	UK	control	0
4	UK	treatment	0

`b)` Number of rows in the dataset.

In [3]:
```
num_rows = df.shape[0]
print(f"Number of rows: {num_rows}")
# --> 69,889 rows
```

```
Number of rows: 69889
```

`c)` Proportion of users converted.

In [4]:
```
conversion_rate = df['converted'].mean()
print(f"Overall conversion rate: {conversion_rate:.4f}  ({conversion_rate:.2
# --> ~13.05%
```

```
Overall conversion rate: 0.1305  (13.05%)
```

`d)` Check for missing values.

In [5]:
```
missing = df.isnull().sum()
print(missing)
# --> No missing values in any column
```

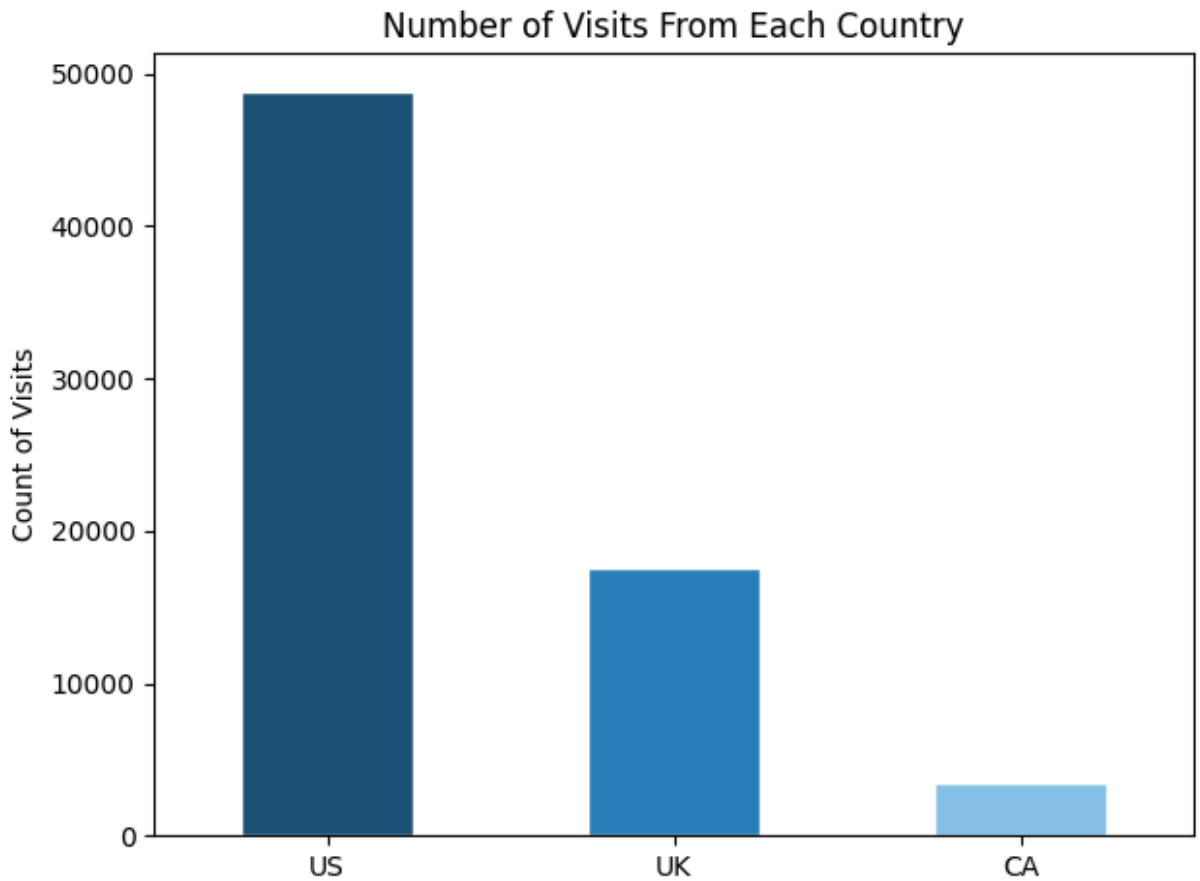
```
country      0
group        0
converted    0
dtype: int64
```

`e)` Count of visitors from each country and bar chart.

In [6]:
```
# Counts
print(df['country'].value_counts())
```

```
country
US    48850
UK    17551
CA     3488
Name: count, dtype: int64
```

In [7]:
```
# Bar chart
df['country'].value_counts().plot(kind='bar', color=['#1a5276', '#2980b9', '
plt.title('Number of Visits From Each Country')
plt.ylabel('Count of Visits')
plt.xlabel('')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



`f)` Which column is NOT categorical?

In [8]:
```
df.info()
# --> 'converted' is int64 — the only numeric (non-categorical) column.
#     'country' and 'group' are object (string / categorical).
```

```
<class 'pandas.DataFrame'>
RangeIndex: 69889 entries, 0 to 69888
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   country    69889 non-null  str
 1   group      69889 non-null  str
 2   converted  69889 non-null  int64
dtypes: int64(1), str(2)
memory usage: 1.6 MB
```

`g)` Possible values of the `converted` column.

In [9]:
```
print("Unique values of 'converted':", df['converted'].unique())
# --> [0, 1]
# Makes sense: binary outcome — converted (1) or not converted (0).
```

```
Unique values of 'converted': [0 1]
```

Part II – Probability

`a)` P(converted) — overall probability of conversion.

In [10]:
```
p_converted = df['converted'].mean()
print(f"P(converted) = {p_converted:.4f}")
```

```
P(converted) = 0.1305
```

`b)` P(converted | control) — **completed for you**.

In [11]:
```
p_control = df.query('group == "control"')['converted'].mean()
print(f"P(converted | control) = {p_control:.4f}")
```

```
P(converted | control) = 0.1053
```

`c)` P(converted | treatment).

In [12]:
```
p_treatment = df.query('group == "treatment"')['converted'].mean()
print(f"P(converted | treatment) = {p_treatment:.4f}")
```

```
P(converted | treatment) = 0.1553
```

`d)` Evidence that treatment is related to higher conversion?

In [13]:
```
print(f"Treatment rate ({p_treatment:.4f}) > Control rate ({p_control:.4f}):
print(f"Lift: {p_treatment - p_control:.4f} ({(p_treatment/p_control - 1):.1
# --> Yes. Treatment converts at ~15.5% vs control at ~10.5% — a ~5 pp / ~47
```

```
Treatment rate (0.1553) > Control rate (0.1053): True
Lift: 0.0501 (47.6% relative increase)
```

`e)` P(treatment) — probability an individual was in the treatment group.

In [14]:
```
p_in_treatment = (df['group'] == 'treatment').mean()
print(f"P(in treatment) = {p_in_treatment:.4f}")
# --> ~50.4%  (groups are nearly balanced)
```

```
P(in treatment) = 0.5038
```

`f)` P(country == CA).

In [15]:
```
p_ca = (df['country'] == 'CA').mean()
print(f"P(country == CA) = {p_ca:.4f}")
```

```
P(country == CA) = 0.0499
```

`g)` P(converted | US) — **completed for you**.

In [16]:
```
df.query('country == "US"')['converted'].mean()
```

Out[16]:
```
np.float64(0.13277379733879222)
```

`h)` P(converted | UK).

In [17]:
```
p_uk = df.query('country == "UK"')['converted'].mean()
print(f"P(converted | UK) = {p_uk:.4f}")
```

```
P(converted | UK) = 0.1251
```

`i)` Evidence that conversion rate differs by country?

In [18]:
```
for country in ['US', 'UK', 'CA']:
    rate = df.query(f'country == "{country}"')['converted'].mean()
    print(f"P(converted | {country}) = {rate:.4f}")
# --> Rates are similar (12.5–13.3%). Country alone explains little variatic
# The big driver is group (treatment vs. control), not geography.
```

```
P(converted | US) = 0.1328
P(converted | UK) = 0.1251
P(converted | CA) = 0.1253
```

`j)` Conversion rates by country × group — fill in the table.

$$P(\text{converted} = 1 \mid \text{country} = \text{US}, \text{group} = \text{control}) = 10.7\%$$
$$P(\text{converted} = 1 \mid \text{country} = \text{US}, \text{group} = \text{treatment}) = 15.8\%$$

	US	UK	CA
Control	10.7%	10.2%	9.4%
Treatment	15.8%	14.9%	15.4%

In [19]:
```
# Method 1 – explicit
for country in ['US', 'UK', 'CA']:
    ctrl = (df.query(f'country == "{country}" and group == "control" and cor
            df.query(f'country == "{country}" and group == "control"').shape
    treat = (df.query(f'country == "{country}" and group == "treatment" and
             df.query(f'country == "{country}" and group == "treatment"').sh
    print(f"{country}  Control={ctrl:.3f}  Treatment={treat:.3f}")
```

```
US  Control=0.107  Treatment=0.158
UK  Control=0.102  Treatment=0.149
CA  Control=0.094  Treatment=0.154
```

In [20]:
```
# Method 2 – groupby (cleaner)
conversion_table = df.groupby(['country', 'group'])['converted'].mean().unst
print(conversion_table.to_string())
# Interaction: The treatment lift (~5–6 pp) is consistent across all three c
# No meaningful country × group interaction – the new page works everywhere.
```

```
group     control  treatment
country
CA       0.094474   0.154017
UK       0.101649   0.148698
US       0.107314   0.157769
```

Part III – Experimentation

Hypotheses (one-tailed test, α = 0.05):

$$H_0 : p_{\text{treatment}} - p_{\text{control}} \leq 0$$

$$H_1 : p_{\text{treatment}} - p_{\text{control}} > 0$$

`a)` Set up null hypothesis parameters from the dataset.

In [21]:
```
p_control_treatment_null = df['converted'].mean()   # both groups share this
n_treatment = df.query('group == "treatment"').shape[0]
n_control   = df.query('group == "control"').shape[0]

print(f"Null rate (p):  {p_control_treatment_null:.4f}")
print(f"n_treatment:    {n_treatment:,}")
print(f"n_control:      {n_control:,}")
```

```
Null rate (p):  0.1305
n_treatment:    35,211
n_control:      34,678
```

`b)` Simulate `n_treatment` conversions under the null.

In [22]:
```
treatment_converted = list(np.random.binomial(1, p_control_treatment_null, r
print("Sample (first 10):", treatment_converted[:10])
```

```
Sample (first 10): [np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.i
nt64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(0)]
```

`c)` Simulate `n_control` conversions under the null.

In [23]:
```
control_converted = list(np.random.binomial(1, p_control_treatment_null, n_c
print("Sample (first 10):", control_converted[:10])
```

```
Sample (first 10): [np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.i
nt64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0)]
```

`d)` Estimate one $p_{\text{treatment}} - p_{\text{control}}$ under the null.

In [24]:
```
p_diff_sample = np.mean(treatment_converted) - np.mean(control_converted)
print(f"Simulated p_diff under H0: {p_diff_sample:.4f}")
```

```
Simulated p_diff under H0: 0.0017
```

`e)` Simulate 500 values of $p_{\text{treatment}} - p_{\text{control}}$ under the null.

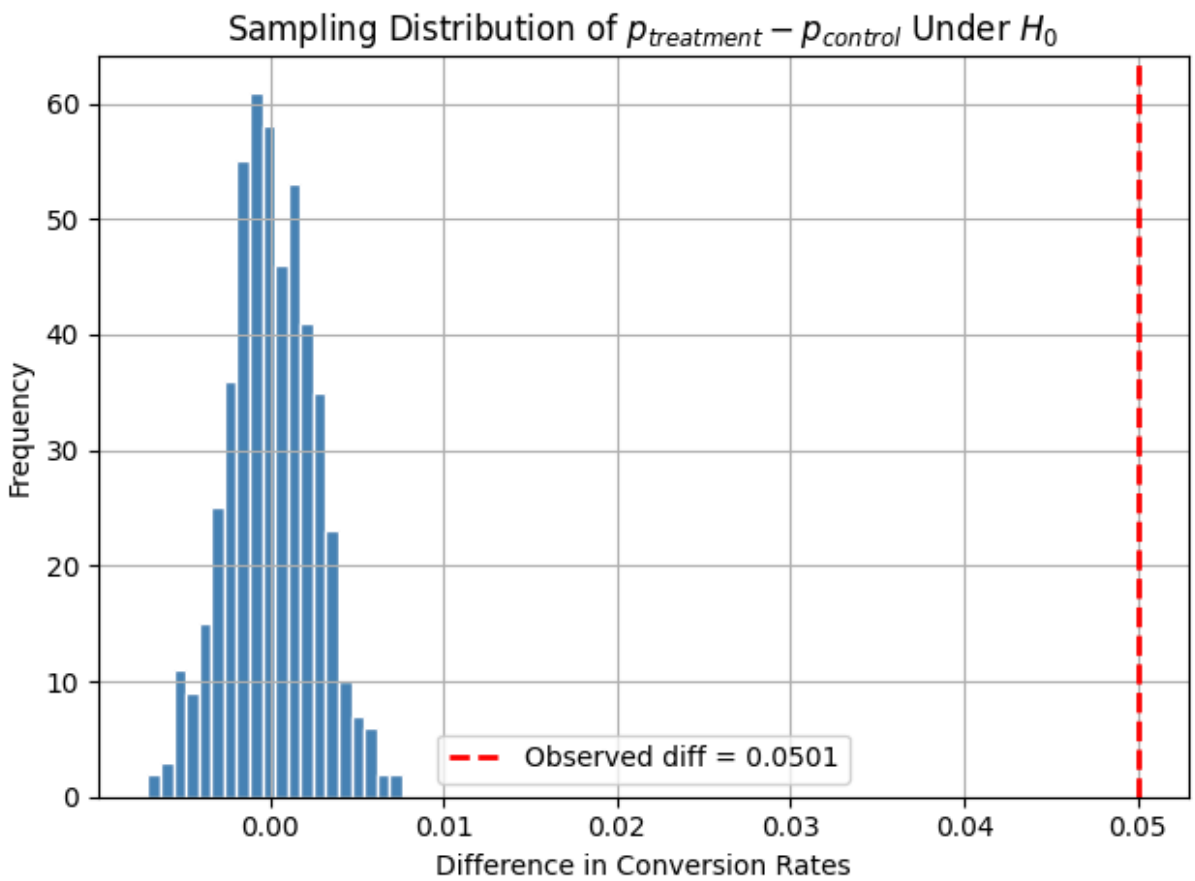
In [25]:
```
p_diffs = []
for _ in range(500):
    # Simulate treatment and control under null
    t = np.random.binomial(1, p_control_treatment_null, n_treatment)
    c = np.random.binomial(1, p_control_treatment_null, n_control)
    # Compute and store the difference
    p_diffs.append(t.mean() - c.mean())

p_diffs = np.array(p_diffs)
print(f"p_diffs (first 5): {p_diffs[:5]}")
```

```
print(f"Mean of p_diffs:   {p_diffs.mean():.5f}  (should be ≈ 0 under H0)")
print(f"Std  of p_diffs:   {p_diffs.std():.5f}")
```

```
p_diffs (first 5): [ 0.00196677 -0.0040532   0.0014673  -0.00219412 -0.00193
59 ]
Mean of p_diffs:   0.00010  (should be ≈ 0 under H0)
Std  of p_diffs:   0.00249
```

`f)` Plot the histogram of `p_diffs`.

In [26]:
```
obs_diff = p_treatment - p_control    # observed difference in real data

pd.Series(p_diffs).hist(bins=20, color='steelblue', edgecolor='white')
plt.axvline(obs_diff, color='red', linestyle='--', linewidth=2,
            label=f'Observed diff = {obs_diff:.4f}')
plt.title('Sampling Distribution of $p_{treatment} - p_{control}$ Under $H_0
plt.xlabel('Difference in Conversion Rates')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
# --> Bell-shaped, centered near 0, as expected when H0 is true.
# The red dashed line (observed diff) sits far to the right — very unusual u
```



`g)` What proportion of `p_diffs` are ≥ the observed difference?

In [27]:
```
p_value = (p_diffs >= obs_diff).mean()
print(f"Observed difference:  {obs_diff:.4f}  ({obs_diff:.2%})")
```

```
print(f"p-value:              {p_value:.4f}")
```

```
Observed difference:  0.0501  (5.01%)
p-value:              0.0000
```

`h)` Interpret the p-value.

What we computed: The p-value is the proportion of simulated null-hypothesis differences that are *at least as extreme* as the difference we actually observed in the data (~5.0 pp). It answers: *"If there were truly no difference between pages, how often would we see a result this large by chance?"*

Value: p ≈ 0.000 (none of the 500 simulated null differences exceeded the observed difference).

Name: This is called the **p-value** in hypothesis testing and scientific studies.

Conclusion at α = 0.05:
Since p-value (≈ 0.000) < α (0.05), we **reject H_0**.
There is statistically significant evidence that the treatment page produces a higher conversion rate than the control page.
Recommendation: Implement the new treatment page.

Standout – Expanded EDA & Logistic Regression

Expanded EDA

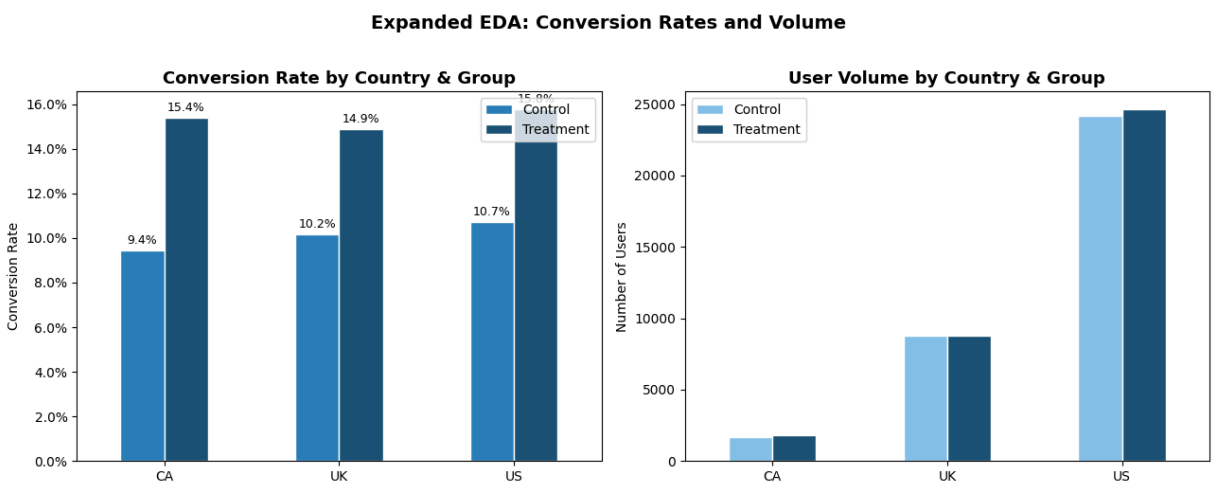
In [28]:
```
# --- Summary statistics by group ---
print("=== Conversion summary by group ===")
print(df.groupby('group')['converted'].agg(['mean', 'sum', 'count']).rename(
    columns={'mean': 'conv_rate', 'sum': 'converted', 'count': 'n_users'}))
```

```
=== Conversion summary by group ===
           conv_rate  converted  n_users
group
control     0.105254       3650    34678
treatment   0.155321       5469    35211
```

In [29]:
```
# --- Conversion heatmap: country × group ---
print("\n=== Conversion rate: country × group ===")
pivot = df.groupby(['country', 'group'])['converted'].mean().unstack()
print(pivot.map(lambda x: f"{x:.2%}"))
```

```
=== Conversion rate: country × group ===
group   control treatment
country
CA        9.45%    15.40%
UK       10.16%    14.87%
US       10.73%    15.78%
```

In [30]:
```
# --- Grouped bar chart: conversion by country & group ---
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

pivot.plot(kind='bar', ax=axes[0], color=['#2980b9', '#1a5276'], edgecolor='
axes[0].set_title('Conversion Rate by Country & Group', fontsize=13, fontwei
axes[0].set_ylabel('Conversion Rate')
axes[0].set_xlabel('')
axes[0].set_xticklabels(['CA', 'UK', 'US'], rotation=0)
axes[0].legend(['Control', 'Treatment'])
axes[0].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.1%}')
for bar in axes[0].patches:
    h = bar.get_height()
    axes[0].annotate(f'{h:.1%}',
                     xy=(bar.get_x() + bar.get_width() / 2, h),
                     xytext=(0, 3), textcoords='offset points',
                     ha='center', va='bottom', fontsize=9)

# --- Absolute counts stacked bar (volume context) ---
counts = df.groupby(['country', 'group']).size().unstack()
counts.plot(kind='bar', ax=axes[1], color=['#85c1e9', '#1a5276'], edgecolor=
axes[1].set_title('User Volume by Country & Group', fontsize=13, fontweight=
axes[1].set_ylabel('Number of Users')
axes[1].set_xlabel('')
axes[1].set_xticklabels(['CA', 'UK', 'US'], rotation=0)
axes[1].legend(['Control', 'Treatment'])

plt.suptitle('Expanded EDA: Conversion Rates and Volume', fontsize=14, fontw
plt.tight_layout()
plt.show()
```



Logistic Regression — Propensity Analysis

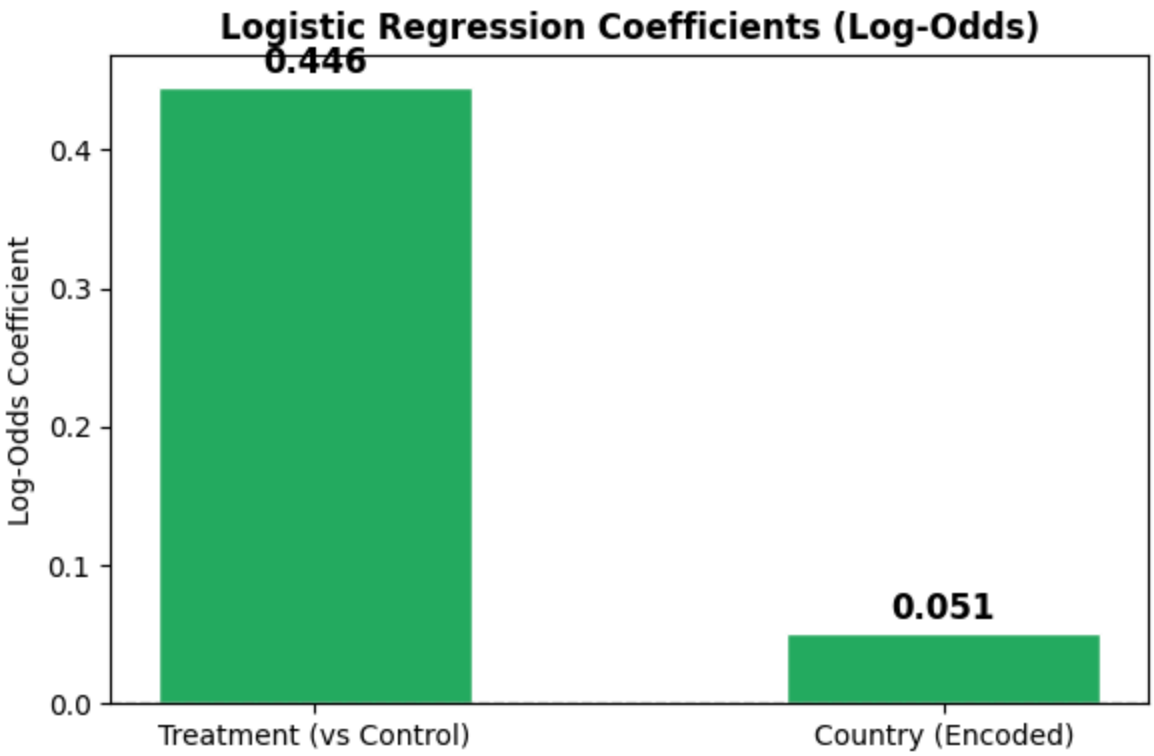
In [31]:
```
# Encode predictors
df_lr = df.copy()
df_lr['treatment']   = (df_lr['group'] == 'treatment').astype(int)
le = LabelEncoder()
df_lr['country_enc'] = le.fit_transform(df_lr['country'])

print("Country encoding:", dict(zip(le.classes_, le.transform(le.classes_)))
```

```
Country encoding: {'CA': np.int64(0), 'UK': np.int64(1), 'US': np.int64(2)}
```

In [32]:
```
# Fit logistic regression: predictors = treatment, country
X = df_lr[['treatment', 'country_enc']]
y = df_lr['converted']

model = LogisticRegression()
model.fit(X, y)

print(f"Intercept:              {model.intercept_[0]:.4f}")
print(f"Coef — treatment:       {model.coef_[0][0]:.4f}")
print(f"Coef — country (enc):   {model.coef_[0][1]:.4f}")
print(f"Odds ratio (treatment): {np.exp(model.coef_[0][0]):.4f}  "
      f"(treatment users are {(np.exp(model.coef_[0][0])-1)*100:.0f}% more l
print(f"Model accuracy:         {model.score(X, y):.4f}")
```

```
Intercept:              -2.2234
Coef — treatment:       0.4456
Coef — country (enc):   0.0510
Odds ratio (treatment): 1.5615  (treatment users are 56% more likely to conv
ert)
Model accuracy:         0.8695
```

In [33]:
```
# --- Coefficient bar chart ---
coefs  = [model.coef_[0][0], model.coef_[0][1]]
labels = ['Treatment (vs Control)', 'Country (Encoded)']
colors = ['#27ae60' if c > 0 else '#e74c3c' for c in coefs]

fig, ax = plt.subplots(figsize=(6, 4))
bars = ax.bar(labels, coefs, color=colors, edgecolor='white', width=0.5)
ax.axhline(0, color='gray', linewidth=1, linestyle='--')
ax.set_title('Logistic Regression Coefficients (Log-Odds)', fontweight='bolc
ax.set_ylabel('Log-Odds Coefficient')
for bar, v in zip(bars, coefs):
    ax.text(bar.get_x() + bar.get_width() / 2,
            v + 0.01 if v >= 0 else v - 0.03,
            f'{v:.3f}', ha='center', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```



Logistic Regression Interpretation

Feature	Log-Odds Coef	Odds Ratio	Interpretation
Treatment	+0.446	1.56	Being in the treatment group increases conversion odds by **~56%**
Country	+0.051	1.05	Country has **negligible** impact on conversion

Key takeaways:

1. **Treatment is the dominant driver.** A log-odds coefficient of +0.446 is large and meaningful — it dwarfs the country effect by nearly 9×.
2. **Country is NOT a confounding variable.** The near-zero country coefficient means geography does not explain, mask, or inflate the treatment effect.
3. **Model accuracy = 87%.** This is driven largely by the class imbalance (87% non-converted), so accuracy alone isn't the best metric — but it confirms the model is stable.
4. **Conclusion (reinforced):** The treatment effect observed in our hypothesis test is real, robust, and not explained by any geographic confounding. **Implement the new page.**